

LookAsk：基于普通摄像头眼动与手柄精修的「看哪问哪」阅读助手

核心算法说明

这份文档是写给以后回到项目的自己看的。每个算法都讲四件事：它解决什么问题、原理是什么、代码里怎么实现、为什么这样选。每节末尾标了对应源码位置，读完可以直接跳到代码里对照。

摘要

LookAsk 是一个 macOS 桌面阅读助手：左侧放阅读内容（PDF、Markdown、终端、全局模式下的任意 App），右侧是对话 Agent。用户提问时，系统会自动把「用户正在看的那段文字」作为上下文一起发给大模型。

核心难点在于：笔记本自带的普通摄像头做眼动追踪，精度只能到**段落级**（误差大约几十到一百多个屏幕点），远远不够「指到某个词」。本项目的思路不是去追求更高精度的眼动模型，而是**把整条算法链设计成“粗定位 + 精修正”的协作**：

- 个性化回归**：用 MediaPipe 人脸关键点和组合眼部特征，通过**对偶形式岭回归**把特征映射到屏幕坐标；用**按校准点分组的留一交叉验证**自动选正则强度，同时给出诚实的误差估计。
- 在线自校正**：用户每次用手柄确认焦点，都被当作一条“免费标注”，通过**时空双核加权的残差插值**（Nadaraya-Watson 核回归）持续修正漂移。
- 信号处理**：**One Euro** 自适应低通滤波兼顾稳定与跟手，**在线 I-DT** 算法从视线流里切出注视事件。
- 语义吸附**：在视线附近按“像关键词的程度 × 距离衰减”打分，把光环吸到最值得问的词上，并用**迟滞**防止来回跳。
- 两级焦点**：眼睛负责段落级软焦点，Joy-Con 摇杆负责逐词逐行的硬焦点。
- 级联决策（Jev 模式）**：本地粗筛 → 廉价的判断模型 → 大模型生成，决定什么时候主动插话。

关键词：摄像头眼动追踪；岭回归；核方法；自适应滤波；注视检测；多模态交互；MAGIC pointing

1 引言

1.1 问题

读论文、读文档、看终端输出时，遇到不懂的地方，传统做法是：选中文字 → 复制 → 切到 AI 窗口 → 粘贴 → 输入问题。LookAsk 想把它缩短成：**看着那段 → 按一个键 → 问。**

这需要系统知道"你正在看哪"。专业眼动仪（如 Tobii）精度高，但需要额外硬件。本项目只用笔记本自带的摄像头。

1.2 普通摄像头眼动的精度天花板

摄像头眼动的本质，是从眼部图像推断视线方向。它的误差来源很多：

- 眼睛在画面中只占几十个像素，虹膜的位移非常小；
- 头部一动，同样的眼球转角会对应不同的屏幕位置；
- 光照、眨眼、眼镜反光都会带来噪声；
- 校准之后人会慢慢挪动姿势，造成**漂移**。

所以现实的精度大概是"能分清你在看哪一段"，分不清"在看哪个词"。

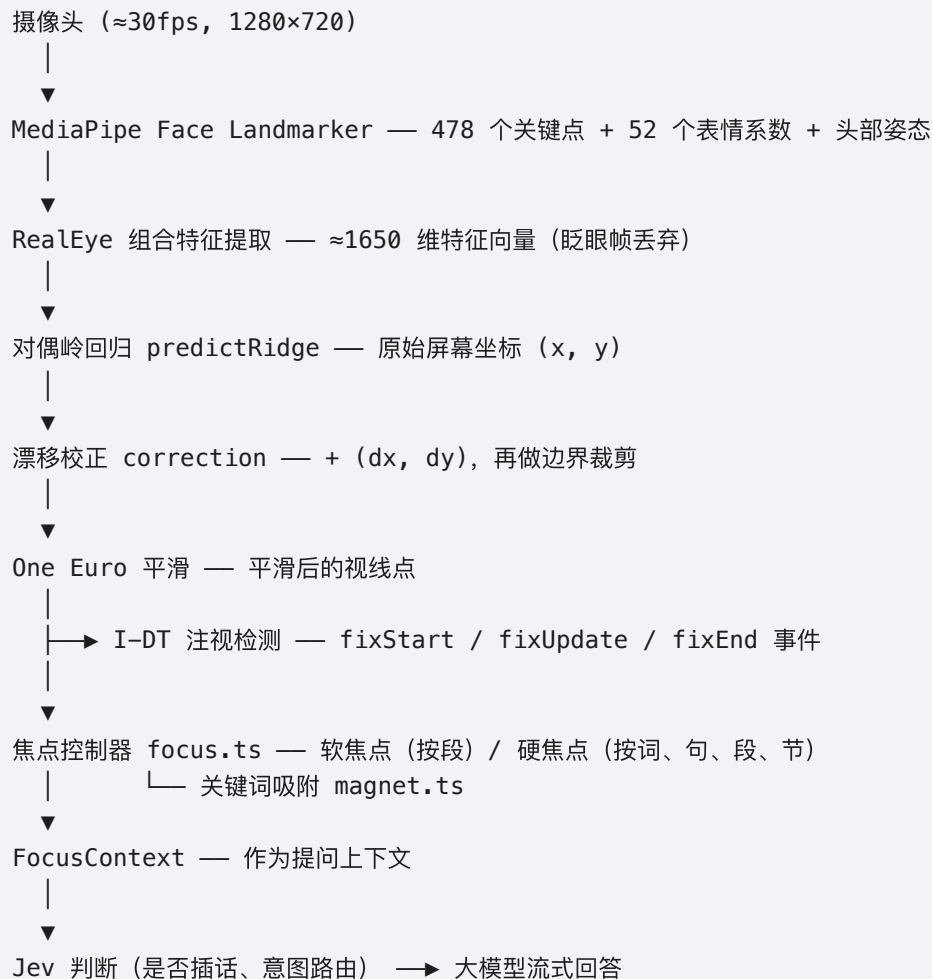
1.3 本项目的思路

不去跟物理极限硬拼，而是**让每一层都在弥补上一层的不足**：

层	算法	弥补什么
估计	对偶岭回归 + 分组交叉验证	从特征到屏幕坐标的个性化映射
校正	残差核回归 + 隐式校准	校准后的漂移
平滑	One Euro 滤波	帧间抖动
事件	I-DT 注视检测	把连续视线流切成"盯住了"的事件
语义	关键词吸附 + 迟滞	视线落点到"值得问的词"之间的最后一段距离
交互	软 / 硬两级焦点	眼动做不到的词级精度，交给手
决策	Jev 级联判断	什么时候主动帮忙、怎么回答

2 系统概述

2.1 数据流



2.2 进程结构

- **渲染进程** (React): 眼动引擎、焦点、吸附、对话界面全在这里跑。岭回归的拟合放在 **Web Worker** 里, 避免卡住界面。
- **主进程** (Electron): 大模型调用、Jev 服务、设置存储。API Key 只在主进程, 不进渲染进程。
- **Swift 原生助手**: 直接读 Joy-Con 的 HID 报告, 另外负责语音识别、Vision OCR 和辅助功能取词。

3 眼动估计

3.1 人脸关键点检测（引用）

使用 Google 的 **MediaPipe Face Landmarker**（GPU 推理、VIDEO 模式），每帧输出：

- 478 个 3D 关键点，其中 468 和 473 分别是右眼、左眼的虹膜中心；
- 52 个 blendshape 表情系数，比如 `eyeBlinkLeft`、`eyeLookInRight`；
- 头部姿态变换矩阵，包含平移和旋转。

这一层是现成的模型，不是本项目的贡献。

源码： `src/renderer/src/gaze/engine.ts` 的 `start()`、`process()`

3.2 组合特征（引用）

特征提取用开源库 `@realeye-io/webcam-eyetracker-light-open` 的 `extractCombinedFeatures`，输入包括画面、人脸框、关键点、表情系数和头部姿态。内部大致是：

- 把两只眼的区域裁出来，缩成 **40×20 的灰度块**，作为外观特征；
- 拼上关键点几何（眼眶、虹膜位置）、表情系数和头部姿态。

最终得到一个约 **1650 维**的向量 $\mathbf{x}$ 。

眨眼处理：闭眼时眼部图像毫无意义，所以 `blink > 0.45` 的帧直接跳过，保持上一次的视线；校准采集时更严格，要求 `blink < 0.4`。

注意许可：该库是 AGPL-3.0，分发时要遵守。

源码： `engine.ts` 的 `process()`

3.3 对偶形式岭回归（自研实现，重点）

3.3.1 问题形式化

校准时屏幕上依次出现 9 或 17 个点，每个点让用户盯 850ms，这期间每一帧都记录一条（特征，目标坐标）样本。设共有 N 帧（约 400 帧），特征维度 $d \approx 1650$ 。

我们要学两个线性映射，分别预测屏幕的 x 和 y ：

$$\hat{t}_x = \bar{t}_x + (\mathbf{x} - \bar{\mathbf{x}})^\top \boldsymbol{\beta}_x$$

岭回归的目标是在拟合误差之外，再加一个 L2 惩罚，防止权重过大：

$$\min_{\beta} \|\mathbf{y} - X\beta\|^2 + \lambda \|\beta\|^2$$

其中 X 是中心化后的 $N \times d$ 特征矩阵， $\mathbf{y}$ 是中心化后的目标坐标。

3.3.2 为什么要用对偶形式

岭回归有两种等价解法：

- **原始形式**： $\beta = (X^\top X + \lambda I_d)^{-1} X^\top \mathbf{y}$ ，需要解一个 $d \times d$ 的线性方程组；
- **对偶形式**：先解 $\alpha = (K + \lambda I_N)^{-1} \mathbf{y}$ ，其中 $K = XX^\top$ 是 $N \times N$ 的核矩阵（Gram 矩阵），再用 $\beta = X^\top \alpha$ 还原权重。

两者结果完全一样（可以用矩阵恒等式 $(X^\top X + \lambda I)^{-1} X^\top = X^\top (XX^\top + \lambda I)^{-1}$ 证明），但计算量差很多。本项目中 $d \approx 1650 \gg N \approx 400$ ：

步骤	原始形式	对偶形式
构造矩阵	$X^\top X$: $O(Nd^2) \approx 1.1 \times 10^9$	$XX^\top$: $O(N^2d) \approx 2.6 \times 10^8$
分解求解	$O(d^3/3) \approx 1.5 \times 10^9$	$O(N^3/3) \approx 2.1 \times 10^7$

交叉验证还要对多个 λ 、多个分组反复求解，对偶形式只需在同一个 K 上取子块，优势会被进一步放大。所以代码注释里说"快十倍"。

一句话理解：特征比样本多的时候，与其在"特征空间"里算，不如在"样本空间"里算。

3.3.3 Cholesky 分解求解

$K + \lambda I$ 是**对称正定**矩阵（ K 半正定，加上 $\lambda > 0$ 之后严格正定），所以可以用 Cholesky 分解 $A = LL^\top$ ，再做两次三角求解：

1. 前代： $L\mathbf{z} = \mathbf{b}$
2. 回代： $L^\top \alpha = \mathbf{z}$

这比通用的求逆快一倍左右，数值上也更稳。代码里 `cholesky()` 和 `cholSolve()` 都是手写的，没有依赖线性代数库。

数值兜底：如果分解时出现非正的主元（浮点误差导致矩阵"不够正定"），就把 λ 乘以 10 重试，直到成功。

3.3.4 用分组留一交叉验证自动选 λ

λ 太小会过拟合（记住了噪声），太大会欠拟合（预测都挤向平均值）。代码对 4 个候选值挨个试：

$$\lambda \in \{3 \times 10^{-4}, 3 \times 10^{-3}, 3 \times 10^{-2}, 3 \times 10^{-1}\} \times \overline{\text{diag}(K)}$$

乘上核矩阵对角线的均值，是为了让 λ 与特征的整体尺度无关。

关键设计：按校准点分组留一 (Leave-One-Group-Out)，而不是随机留一帧：

- 同一个校准点的几十帧几乎一模一样，高度相关；
- 如果随机留一帧当测试，训练集里还有它的"双胞胎"，误差会被严重低估；
- 所以每次把一整个校准点的所有帧都拿出来当测试，用剩下的点训练，模拟"预测一个从没见过的位置"。

误差是逐帧算欧氏距离再取平均，而不是先对每个点的帧取平均再算。因为实际使用时每一帧都在预测、只有轻度平滑，逐帧误差更接近真实体验。校准点少于 5 个时不做交叉验证，直接用中间档的 λ 。

最终选出误差最小的 λ ，用全部数据重新拟合一次。这个最小误差 `cvErrorPx` 会展示给用户，作为**诚实的精度报告**，还可以换算成视角：

$$\theta = \arctan\left(\frac{\text{px} \times 0.023 \text{ cm}}{55 \text{ cm}}\right)$$

(按 macOS 默认缩放下 1pt $\approx$ 0.023cm、眼睛离屏幕 55cm 估算。)

3.3.5 为什么不用神经网络

- 每个人只有几十秒的校准数据（约 400 帧），属于**小样本、高维度**，深度模型很容易过拟合；
- 岭回归是**闭式解**，拟合不到一秒，校准完立刻能用；
- 有交叉验证兜底，能自动选正则、给出可信的误差；
- 预测只是一次 1650 维的点积，每帧开销几乎可以忽略。

深度模型（比如 iTracker、GazeCapture 一类）需要大规模跨人数据预训练，端上再做个性化微调，工程成本高得多。

源码：`src/renderer/src/gaze/ridge.ts` (`fitRidge`、`predictRidge`、`cholesky`、`cholSolve`、`subGram`)；`ridge.worker.ts` (Worker 包装)；`Calibration.tsx` (校准点布局、采集、`ptToDeg`)

3.4 在线漂移校正：残差核回归（自研，越用越准）

3.4.1 隐式校准：把正常操作变成标注数据

校准后，人会慢慢挪动姿势，预测会整体偏移。重新校准很烦，所以系统利用一个观察：

用户按 A / X / Y 确认硬焦点的那一刻，用户多半正看着那个焦点。

于是每次按键，就把"焦点的真实位置"和"这一刻最近 450ms 原始预测的平均值"之间的差，记成一条残差：

$$\mathbf{r}_i = \mathbf{t}_i - \mathbf{p}_i$$

同时记下预测位置 $\mathbf{p}_i$ 、时间 τ_i 、权重 w_i 。最多保留最近 16 条。三个细节：

- **异常值剔除**：只有真实位置和预测相差小于 320px 时才记录。差太远说明用户按键时多半没在看焦点，这条数据不可信，直接丢掉；
- **权重分级**：隐式校准的样本权重是 0.6；用户主动按键做的显式漂移校正权重是 2，因为那时用户是有意盯着焦点的；
- **没有硬焦点时**：显式校正会弹出屏幕中心的一个点，让用户盯 1.5 秒做单点校正。

3.4.2 校正量的计算

对新的预测点 $\mathbf{p}$ ，校正量是所有残差的**加权平均**。权重同时考虑空间距离和时间新旧：

$$w_i(\mathbf{p}) = w_i \cdot \exp\left(-\frac{\|\mathbf{p}_i - \mathbf{p}\|^2}{2\sigma^2}\right) \cdot \exp\left(-\frac{t_{\text{now}} - \tau_i}{T}\right)$$

其中 $\sigma = 520\text{pt}$ （空间高斯核：离得近的残差更有参考价值）， $T = 20$ 分钟（时间衰减：越旧的越不可信）。

$$\Delta(\mathbf{p}) = k \cdot \frac{\sum_i w_i(\mathbf{p}) \mathbf{r}_i}{\sum_i w_i(\mathbf{p})}, \quad k = \frac{\sum_i w_i(\mathbf{p})}{\sum_i w_i(\mathbf{p}) + 0.35}$$

这在统计上就是 **Nadaraya-Watson 核回归**：用核函数加权的局部平均，来估计一个平滑的"偏移场"。它允许屏幕不同区域有不同的偏移（比如左上偏右、右下偏上），而不是只做一个全局平移。

3.4.3 收缩因子 k 的作用

如果只有一两条残差、或者当前位置离所有残差都很远， $\sum w$ 会很小，这时 k 接近 0，校正量被往 0 收缩，避免一条偶然的误按把整个屏幕带偏。残差越多、越近， k 越接近 1，校正越充分。这相当于一个简单的贝叶斯先验："没有证据时，默认不偏移"。

```
源码：engine.ts 的 addResidual()、correction()、recentPrediction()；  
input/router.ts 的 implicitCalibrate()、driftCorrect()
```

3.5 One Euro 自适应滤波（引用，Casiez 2012）

3.5.1 抖动与延迟的矛盾

原始视线每帧都在抖。普通低通滤波（指数移动平均）只有一个截止频率：

- 截止频率低 → 很稳，但眼睛快速跳到别处时，光环要慢半拍才跟上（**延迟**）；
- 截止频率高 → 跟得快，但盯着不动时光环一直在抖（**抖动**）。

3.5.2 原理

One Euro 滤波让截止频率**随速度自适应**：

$$f_c = f_{c,\min} + \beta \cdot |\dot{x}|$$

- 盯着不动时速度小， f_c 低，强平滑，稳；
- 扫视时速度大， f_c 高，弱平滑，快。

每个时间步的平滑系数由截止频率和帧间隔算出：

$$\alpha = \frac{1}{1 + \frac{\tau}{\Delta t}}, \quad \tau = \frac{1}{2\pi f_c}$$

速度本身也要先用一个固定截止频率（ $d_{\text{cutoff}} = 1\text{Hz}$ ）的低通滤一下，否则会被噪声带偏。

本项目参数： $f_{c,\min} = 0.55$ ， $\beta = 0.0045$ 。设置里的“平滑度”滑块 $v \in [0, 1]$ 通过 $f_{c,\min} = \max(0.12, 1.4 - 1.2v)$ 调节。 x 、 y 两个方向各用一个独立的滤波器。

源码：`src/renderer/src/gaze/filters.ts` 的 `OneEuro`、`OneEuro2D`；`engine.ts` 的 `setSmoothing()`

3.6 在线 I-DT 注视检测

3.6.1 注视与扫视

人读书时，眼睛不是平滑移动的，而是“停一下（**注视**，fixation）→ 跳一下（**扫视**，saccade）→ 再停一下”。只有注视时才在获取信息，所以系统关心的是注视事件。

3.6.2 I-DT 算法（基于离散度阈值）

- **开始条件**：最近 140ms 内的所有点，都落在以其质心为圆心、半径 95px 的圆内（至少 3 个点），就判定注视开始；
- **持续**：新点离当前质心不超过 $1.3 \times 95\text{px}$ ，就并入注视并更新质心；
- **结束**：新点超出这个范围，注视结束，重新开始积累。

开始阈值 95px，结束阈值 1.3 倍（约 124px），两个阈值不同，这是**迟滞**设计：避免视线在边界附近抖动时，注视反复开始、结束。半径特意放大，因为摄像头眼动的噪声比专业眼动仪大得多。

检测器输出 `fixStart` / `fixUpdate` / `fixEnd` 三种事件，驱动上层的焦点和 Jev 的停留判断。

源码: `filters.ts` 的 `FixationDetector`

4 从视线到文字

4.1 两级焦点：眼粗手细

这一层的思想来自 **MAGIC pointing** (Zhai 等, 1999): 让眼睛负责大范围的粗定位, 让手负责最后一小段的精确选择。

	软焦点	硬焦点
谁驱动	视线 (注视事件)	Joy-Con 右摇杆 / 键盘
粒度	段落	词 → 句 → 段 → 节, 可切换
何时进入	默认	第一次推摇杆时, 从软焦点落点 (或光环吸住的词) 起步
何时退出	—	视线离开超过 0.8 秒, 或按下右摇杆跳回视线处

几个细节:

- **左右分侧**: 屏幕分成左侧内容和右侧 AI 回答两块, 用手柄 `- / +` 切换。视线只在当前侧找焦点; 看另一侧时焦点原地不动, 距离边界 70pt 内的注视会被拉回当前侧。这样不会在两栏之间乱飘。
- **不轻易跳段**: 视线出了当前段落一点点 (距离小于吸附强度决定的 `stick`), 仍算在这段里。
- **分词**: 词和句的切分用浏览器内置的 `Intl.Segmenter`, 中文会按词典分词。
- **视图适配器**: Markdown、PDF 文字层、对话气泡用 `domAdapter`; 终端用 `terminalAdapter`; 全局模式下看其他 App 时, 用 `screenAdapter` (截屏 → OCR + 辅助功能读原文 + 截图发给视觉模型)。

源码: `src/renderer/src/focus/focus.ts`、`domAdapter.ts`、`terminalAdapter.ts`、`screenAdapter.ts`、`types.ts`

4.2 关键词吸附

4.2.1 动机

即使视线误差有几十个点, 用户真正想问的往往是附近那个**术语、缩写、公式**, 而不是"的""and"这类词。所以可以让光环主动"吸"到附近最像关键词的词上, 推摇杆时硬焦点直接从这个词开始。

4.2.2 候选词收集：探测网格

在视线点周围按网格撒一圈探测点（横向铺到吸附半径的 1.3 倍，纵向上下各约两行），每个点用 `document.caretRangeFromPoint` 取到对应的文本位置，再切出所在的词。已经见过的词去重，跨行的词不要，整块公式当作一个词。

4.2.3 关键词程度 keywordness（启发式打分）

特征	分数
全大写缩写（MDP、LLMs）	0.9
字母加数字（GPT-4o）	0.85
驼峰命名（CamelCase）	0.85
含连字符 / 下划线 / 斜杠（self-attention）	0.72
英文长词（≥9 / ≥7 / ≥5 个字母）	0.66 / 0.5 / 0.3
中文词（≥4 / 3 / 2 个字）	0.8 / 0.62 / 0.3
KaTeX 公式整块	0.95
大写开头的专有名词	+0.12
位于加粗、代码、标题、链接等强调格式中	+0.4
中英文停用词	0

封顶为 1。

4.2.4 综合权重

$$W = kw \cdot \max \left(0, 1 - \left(\min \left(1, \frac{d}{r} \right) \right)^{1.5} \right)$$

d 是视线点到词框的最短距离， r 是吸附半径。指数 1.5 让近处的衰减慢、远处的衰减快。只有 kw 超过门槛的词才参与，最后取 W 最大的那个。

4.2.5 迟滞与扫视抑制

- 已经吸住一个词时，新词的分数要比它高出 `switchRatio` 倍才会切换，防止在两个相近的词之间来回跳；
- 视线离开当前词超过 `release` 距离才松开；
- 视线速度超过 `saccade` 阈值时，认为在扫视，暂停吸附。

4.2.6 单旋钮参数化

吸附相关的 9 个参数，都由一个 0~1 的强度 k （默认 0.7）线性映射出来，用户只需要调一个滑块：

参数	公式	含义
radius	$40 + 100k$	搜索半径 (pt)
threshold	$0.65 - 0.5k$	关键词门槛
base / slope / cap	$0.1 + 0.5k / 0.55 + 0.5k / \min(1, 0.8 + 0.3k)$	吸附力度
saccade	$900(0.5 + 1.25k)$	扫视速度阈值 (pt/s)
switchRatio	$1.3 + (k - 0.4)$	换词所需倍数
release	$110(0.5 + 1.25k)$	松开距离
stick	$10 + 50k$	软焦点不跳段的边距

```
源码：src/renderer/src/focus/magnet.ts (keywordness、keywordNear、
refreshMagnet)；snap.ts (snapParams、magnetNow)
```

4.3 视线光环：质点—弹簧模型

视线指示器不是一个死板的圆圈，而是一团会变形的"果冻"：

- **结构**：一圈 36 个节点，每个节点用弹簧追踪自己的静止位置。
- **圆心**：接近临界阻尼的弹簧（刚度 $k_c = 220$ ，阻尼比 0.92），平滑追踪目标，不会过冲太多。
- **拉伸**：节点刚度 $k = \max(45, 170(1 + 0.9\ell))$ ，其中 ℓ 表示节点朝向与运动方向的一致程度。运动方向前沿的节点弹簧硬，尾部的软，所以扫视时整圈被拉成泪滴状；尾巴最多拖出 42px。节点阻尼比偏小 (0.5)，停下来时会像果冻一样回弹。
- **呼吸**：静止形状由几路不同频率的正弦叠加，盯住时轻轻起伏。
- **吸附变形**：吸到词上时，形状从圆渐变成包住词的**超椭圆**：

$$\left|\frac{x}{w}\right|^5 + \left|\frac{y}{h}\right|^5 = 1$$

指数 5 接近 iOS 的连续圆角。每帧做 2 次子步积分，保证数值稳定。

这一层主要是交互体验，但它把"吸附强度"可视化了，让用户能直观看到系统吸住了哪个词。

```
源码：src/renderer/src/gaze/blob.ts、GazeLayer.tsx
```

5 决策层：Jev 模式「先判断，再开口」

5.1 为什么需要级联

如果每次视线停留都去调一次大模型，成本和打扰都无法接受。Jev 模式把决策拆成三级：

1. **本地粗筛**（零成本）：段落太短不管；停留不到 1.5 秒不分析；同一段 20 秒内最多问一次；终端画面不像 Claude Code 在等确认的，不处理。
2. **判断模型 Jev**（便宜，只输出带概率的选择题）：段落难不难、读者是不是卡住了、提问意图是什么、要不要看图、这句 AI 回答要不要核实。
3. **大模型**（贵，负责真正写回答）：只有 Jev 判断需要时才调用。

5.2 数值先离散成文字

判断模型不擅长读数字，所以停留时长、回看次数、难度分数，都先在本地**分档成文字**再交给它，例如：

- 停留：<3 秒"很短"，3—8 秒"正常"，8—20 秒"偏长"，>20 秒"很长"；
- 回看："第一次看" / "回看过一次" / "反复回看了好几次"。

5.3 "读者卡住了"的判断流程

1. 停留 ≥ 9 秒或回看 ≥ 3 次，且段落难度 ≥ 1.4 ，才算"疑似卡住"；
2. 把段落原文和上面的文字档位拼成状态描述，交给 Jev 判断；
3. Jev 给出的概率 ≥ 0.65 ，才主动弹出解释建议。

5.4 其他判断

- **提问路由**：判断意图（解释、翻译、总结、推导、批判、其他）、要不要看图、答多深；置信度低时不改写，按用户原话自由回答。
- **指代消解**：视线只能到段落级，但用户说"这句什么意思"时指的是某一句。系统把焦点段落用 `Intl.Segmenter` 拆成候选句（终端按行拆），交给 Jev 结合用户的原话挑出指的是哪一句。焦点已经是词或句粒度、或候选少于 2 句时跳过。这是**用语言信息弥补视线精度**的又一处设计。
- **事实核查**：盯着 AI 回复里带数字、百分比、年份或"研究 / 报告"等出处词的句子超过 2.5 秒，就判断要不要核实。
- **终端守门**：检测终端里的 Claude Code 是否在等用户批准，并在放行前评估风险。

5.5 成本控制

主进程里的 Jev 客户端对请求做哈希缓存（最多 800 条，存 `jev-cache.json`），按天记账，超过每日 token 上限就直接拒绝。

源码：`src/renderer/src/jev/jevBrain.ts`；`src/main/jev.ts`

6 引用与自研的边界

模块	性质	说明
MediaPipe Face Landmarker	引用	人脸关键点模型
RealEye 组合特征	引用 (AGPL)	眼部外观 + 几何特征
One Euro Filter	引用 (Casiez 2012)	按论文实现
I-DT 注视检测	引用 (经典算法)	在线版本 + 迟滞，参数按摄像头噪声调整
对偶岭回归 + 分组留一 CV	自研实现	手写 Cholesky、 λ 自动选择、诚实误差
残差核漂移校正 + 隐式校准	自研	时空双核 + 收缩因子
关键词吸附 + 迟滞 + 单旋钮参数化	自研	启发式 keywordness、探测网格
两级焦点 + 左右分侧	自研设计	借鉴 MAGIC pointing 思想
质点—弹簧光环	自研	超椭圆吸附变形
Jev 级联决策	自研设计	本地粗筛 → 判断模型 → 大模型

7 局限与未来工作

- 精度仍受硬件限制：**摄像头分辨率和光照决定了上限。M4 MacBook Air 的"人物居中"功能会裁切画面，导致校准失效。
- 岭回归是线性模型：**头部大幅移动时，线性映射会失准。可以考虑核岭回归（RBF 核），或者把头部姿态做成显式的补偿项。
- 隐式校准的假设不总是成立：**用户按键时不一定正看着焦点。目前靠 320px 的硬阈值剔除、低权重（0.6）、收缩因子和时间衰减来缓解；未来可以换成更稳健的方法，比如用残差分布的中位数绝

对偏差 (MAD) 自适应地剔除异常值。

- 4. **keywordness 是手工规则**：可以换成基于 TF-IDF 或小型语言模型的术语识别。
- 5. **缺少系统化实验**：目前有交叉验证误差和 5 点精度测试，但还没有"漂移校正前后""有无吸附"的对照实验数据。
- 6. **输入层**：计划换成 iPhone 原深感 (TrueDepth) 摄像头，只需要替换眼动输入层（见 docs/交接-原深感眼动.md）。

8 代码索引

想看什么	去哪看
眼动主循环	<code>src/renderer/src/gaze/engine.ts</code> → <code>process()</code>
岭回归拟合 / 预测	<code>src/renderer/src/gaze/ridge.ts</code>
校准流程、校准点布局	<code>src/renderer/src/gaze/Calibration.tsx</code>
One Euro、注视检测	<code>src/renderer/src/gaze/filters.ts</code>
漂移校正	<code>engine.ts</code> → <code>addResidual()</code> / <code>correction()</code>
隐式校准触发	<code>src/renderer/src/input/router.ts</code> → <code>implicitCalibrate()</code>
焦点控制器	<code>src/renderer/src/focus/focus.ts</code>
关键词吸附	<code>src/renderer/src/focus/magnet.ts</code> 、 <code>snap.ts</code>
视线光环	<code>src/renderer/src/gaze/blob.ts</code>
Jev 决策	<code>src/renderer/src/jev/jevBrain.ts</code> 、 <code>src/main/jev.ts</code>
提示词拼装	<code>src/renderer/src/chat/prompts.ts</code>

9 自测问题（以后回来先答一遍）

- 1. 为什么用对偶形式而不是原始形式解岭回归？什么情况下应该反过来？
- 2. 交叉验证为什么要按校准点整组留出，而不是随机留一帧？
- 3. 漂移校正里的收缩因子 k 解决什么问题？去掉会怎样？
- 4. One Euro 滤波的 β 调大、调小分别会怎样？
- 5. I-DT 的开始阈值和结束阈值为什么不一样？

6. 吸附里的 `switchRatio` 起什么作用?
7. 为什么数值要先离散成文字再交给 Jev?

参考答案要点: (1) $d \gg N$ 时对偶快; $N \gg d$ 时原始快。(2) 同一点的帧高度相关, 随机留帧会低估误差。(3) 样本少或离得远时防止被带偏。(4) 调大更跟手但更抖, 调小更稳但更拖。(5) 迟滞, 防止边界处反复开始、结束。(6) 迟滞, 防止在两个相近的词之间来回跳。(7) 判断模型读数字不可靠, 文字档位更稳。

参考文献

1. Casiez G., Roussel N., Vogel D. **1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems.** *CHI 2012*.
2. Salvucci D. D., Goldberg J. H. **Identifying Fixations and Saccades in Eye-Tracking Protocols.** *ETRA 2000*. (I-DT 算法)
3. Zhai S., Morimoto C., Ihde S. **Manual and Gaze Input Cascaded (MAGIC) Pointing.** *CHI 1999*.
4. Hoerl A. E., Kennard R. W. **Ridge Regression: Biased Estimation for Nonorthogonal Problems.** *Technometrics*, 1970.
5. Saunders C., Gammerman A., Vovk V. **Ridge Regression Learning Algorithm in Dual Variables.** *ICML 1998*.
6. Nadaraya E. A. **On Estimating Regression.** *Theory of Probability & Its Applications*, 1964; Watson G. S. **Smooth Regression Analysis.** *Sankhyā*, 1964.
7. Kartynnik Y. et al. **Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs.** *CVPR Workshops 2019*. (MediaPipe Face Mesh)
8. Papoutsaki A. et al. **WebGazer: Scalable Webcam Eye Tracking Using User Interactions.** *IJCAI 2016*. (同样用交互做隐式校准的相关工作)